

```
import spacy
```

```
# Load English language model
```

```
try:
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
except OSError:
```

```
    import subprocess
```

```
    print("Downloading language model...")
```

```
    subprocess.run(["python", "-m", "spacy", "download", "en_core_web_sm"],  
capture_output=True)
```

```
    nlp = spacy.load('en_core_web_sm')
```

```
def evaluate_answers(answer_file_path, answer_key_file_path):
```

```
    """Compare student answer with answer key and return similarity percentage"""
```

```
    try:
```

```
        # Read files
```

```
        try:
```

```
            with open(answer_file_path, 'r', encoding='utf-8') as f:
```

```
                student_answer = f.read().strip()
```

```
            with open(answer_key_file_path, 'r', encoding='utf-8') as f:
```

```
                answer_key = f.read().strip()
```

```
        except UnicodeDecodeError:
```

```
            with open(answer_file_path, 'r', encoding='latin-1') as f:
```

```
                student_answer = f.read().strip()
```

```
            with open(answer_key_file_path, 'r', encoding='latin-1') as f:
```

```
                answer_key = f.read().strip()
```

```
    if not student_answer or not answer_key:
```

```
        return {
```

```
        "status": "error",  
        "message": "One or both files are empty"  
    }
```

```
# Process with spaCy
```

```
student_doc = nlp(student_answer)
```

```
key_doc = nlp(answer_key)
```

```
# Calculate similarity
```

```
similarity = student_doc.similarity(key_doc) * 100
```

```
return {
```

```
    "status": "success",
```

```
    "overall_score": similarity,
```

```
    "details": [] # Empty details list
```

```
}
```

```
except Exception as e:
```

```
return {
```

```
    "status": "error",
```

```
    "message": f"Error during evaluation: {str(e)}"
```

```
}
```